# Introduction to Scientific Programming

## Lecture 5: *Data Manipulation & Analysis*

Renato Duarte

@Renatocfduarte
comp-neuro.org
@rcfduarte
renato.duarte@cnc.uc.pt

CENTER FOR NEUROSCIENCE AND CELL BIOLOGY
UNIVERSITY OF COIMBRA
PORTUGAL

1 2 9 0 UNIVERSIDADE D COIMBRA

# Part II: *Scientific Computing Core*

## **Day 5**: Data Manipulation & Analysis

**Lecture:**

1. Data analysis pipeline
2. Pandas:
    1. What is pandas?
    2. Pandas core
3. Data analysis with pandas
4. Advanced SciPy (with pandas)
    1. Probability & Statistics
    2. Signal processing
5. Best practices

**Practical:**

- Notebooks, Handout & Tutorial exercises

# Data Analysis Pipeline

The Research Data Workflow

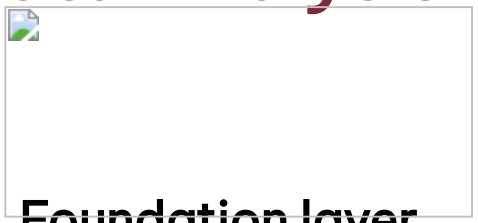
1. **Acquisition**: Instruments generate raw data (images, traces, counts, measurements)
2. **Loading**: Reading data into analysis environment
3. **Cleaning**: Handling missing values, outliers, artifacts
4. **Transformation**: Normalization, scaling, feature extraction
5. **Analysis**: Statistical tests, modeling, pattern discovery
6. **Visualization**: Plots, figures, summaries
7. **Interpretation**: Biological/scientific meaning

**Tools:** `pandas` (data structures and manipulation) + `scipy` (mathematical operations and statistics)

**Goal:** Go from raw tabular data to statistical insights and visualizations

# Data Analysis Pipeline

The tools



**Foundation layer**

- Multidimensional arrays
- Fast numerical operations
- Broadcasting, indexing
- But: no labels, limited missing data handling

**Data structures layer**

- Labeled rows and columns
- Missing data handling (NaN)
- Automatic alignment
- Time series functionality
- Group operations, reshaping

**Scientific algorithms layer**

- Statistics (scipy.stats)
- Signal processing (scipy.signal)
- Optimization, integration
- Linear algebra extensions
- Operates on NumPy arrays

## How they work together

- Pandas DataFrames contain NumPy arrays
- SciPy functions work on pandas columns
- Extract values: `df['col'].values`
- Seamless interoperability

# What is Pandas?

Python Data Analysis Library

**pandas = Python Data Analysis Library**

Open-source library built on NumPy that provides:

- High-performance data structures (Series, DataFrame)
- Tools for reading/writing data in many formats
- Data cleaning and preprocessing capabilities
- Flexible grouping, merging, and reshaping operations
- Time series functionality and missing data handling




*"NumPy provides the arrays, pandas provides the spreadsheet-like interface for working with them."*

pandas provides labeled, relational data structures with intuitive indexing, making it the standard tool for data manipulation and analysis in Python

```python
# Import pandas
import pandas as pd

# Check version
print(pd.__version__)  # '2.3.3'
```

# What is Pandas?

Pandas + Jupyter for interactive analysis

- **Interactive exploration:** Execute code cell-by-cell
- **Immediate feedback:** See results instantly
- **Rich display:** DataFrames render as HTML tables
- **Inline plots:** Visualizations appear directly below code
- **Documentation:** Mix code, results, and explanations
- **Iterative workflow:** Test ideas quickly without full scripts

```python
# Standard imports for data analysis
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Display settings for better output
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
pd.set_option('display.precision', 3)

# Inline plots in Jupyter
%matplotlib inline
```




**Exploratory workflow in Jupyter:**

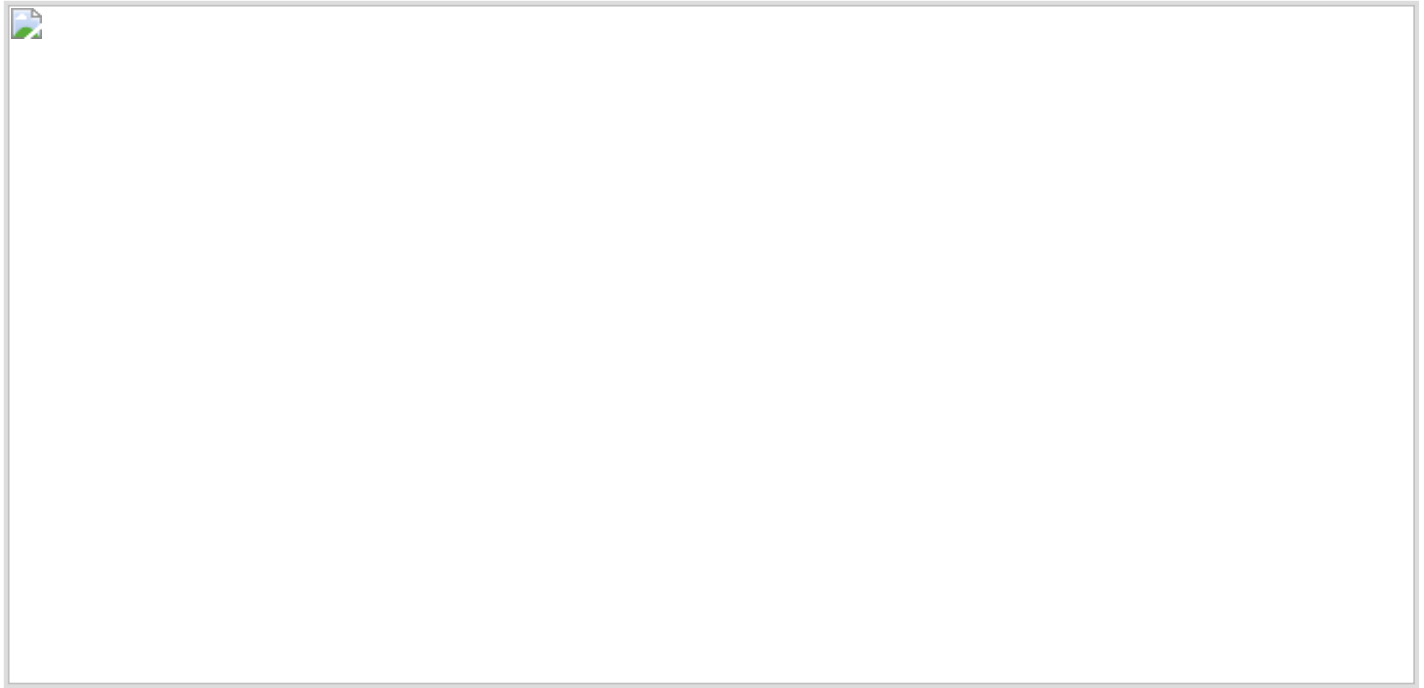
Load data → Inspect structure → Try operations → See immediate results → Adjust approach → Document findings → All in one place

# What is Pandas?

Pandas + Jupyter for interactive analysis

## Rich DataFrame display

Jupyter automatically renders DataFrames as formatted HTML tables:



- Styled headers and index
- Scrollable for large datasets
- Right-aligned numbers
- Easy visual inspection

## Inline plotting

Plots appear directly below code cells:

**Key benefits:** No need to save/open files, instant feedback on data manipulations, easy to spot errors, perfect for iterative exploration and debugging. Code, output, and documentation all in one notebook.

# Pandas Core

## Data Structures: Series and DataFrame

### Series: 1D

```
import pandas

# Create a Ser
s = pd.Series(


# Access by la
s['B']  # 20

# Access by po
s[1]    # 20
```

Series 1

| INDEX | DATA |
|---|---|
| 0 | A |
| 1 | B |
| 2 | C |
| 3 | D |
| 4 | E |
| 5 | F |

&

Series 2

| INDEX | DATA |
|---|---|
| 0 | 1 |
| 1 | 2 |
| 2 | 3 |
| 3 | 4 |
| 4 | 5 |
| 5 | 6 |

&

Series 3

| INDEX | DATA |
|---|---|
| 0 | [1, 2] |
| 1 | A |
| 2 | 1 |
| 3 | (4, 5) |
| 4 | {"a": 1} |
| 5 | 6 |

=

Dataframe

| INDEX | SERIES 1 | SERIES 2 | SERIES 3 |
|---|---|---|---|
| 0 | A | 1 | [1, 2] |
| 1 | B | 2 | A |
| 2 | C | 3 | 1 |
| 3 | D | 4 | (4, 5) |
| 4 | E | 5 | {"a": 1} |
| 5 | F | 6 | 6 |

```
'treated']
```

| Component | |
|---|---|
| **Index** | Row labels (can be integers, strings, dates, etc.) |
| **Columns** | Column labels (names of variables) |
| **Values** | Underlying NumPy array(s) containing the actual data |
| **dtype** | Data type of each column (int64, float64, object, datetime64, etc.) |

**Think of DataFrame as:** A collection of Series (columns) with a shared index, or a spreadsheet with row/column labels

# Pandas Core

## Reading CSV and TSV files

### Basic usage

```
# Basic CSV loading
data = pd.read_csv('experiment_data.csv')

# Tab-separated values
data = pd.read_csv('data.tsv', sep='\t')

# Specify index column
data = pd.read_csv('data.csv', index_col=0)

# Multiple columns as index
data = pd.read_csv('data.csv',
                   index_col=['subject', 'session'])
```

### Common parameters

```
# Skip header rows (e.g., metadata)
data = pd.read_csv('data.csv', skiprows=5)

# Handle comment lines
data = pd.read_csv('data.csv', comment='#')

# Specify columns to read
data = pd.read_csv('data.csv',
                   usecols=['gene', 'expression'])

# Handle missing value codes
data = pd.read_csv('data.csv',
                   na_values=['NA', 'N/A', '.', '--'])
```

### Pro tips:

- Use `nrows=10` to preview large files quickly
- European CSV formats: `decimal=','` and `sep=';'`
- For very large files: `chunksize` parameter for iterative reading

# Pandas Core

## Reading Excel files

**Input Data Frame - df**

| | name | score | subject | class |
|---|---|---|---|---|
| 0 | Krunal | 85 | Maths | 10 |
| 1 | Ankit | 90 | Science | 11 |
| 2 | Dhaval | 78 | History | 10 |

```
df.to_excel("output_with_index.xlsx")
```

appdividend.com

| | name | score | subject | class |
|---|---|---|---|---|
| 0 | Krunal | 85 | Maths | 10 |
| 1 | Ankit | 90 | Science | 11 |
| 2 | Dhaval | 78 | History | 10 |

Output Excel file

```
', 'Metadata'])


)}")
```

| | | | |
|---|---|---|---|
| 2 | Baby Food | 2-day and next-day | 5-7 |
| 3 | Cereal | next-day and 2-day | 8-11 |
| 4 | Fruit | next-day and 2-day | 5-6 |
| 5 | Office Supplies | 2-day and next-day | 7-9 |
| 6 | | | |
| 7 | | | |

| | item_type | priority | shipping_cost |
|---|---|---|---|
| 8 | item_type | priority | shipping_cost |
| 9 | Baby Food | 2-day | $ 5.00 |
| 10 | Baby Food | next-day | $ 7.00 |
| 11 | Cereal | 2-day | $ 8.00 |
| 12 | Cereal | next-day | $ 11.00 |
| 13 | Fruit | 2-day | $ 5.00 |
| 14 | Fruit | next-day | $ 6.00 |
| 15 | Office Supplies | 2-day | $ 7.00 |
| 16 | Office Supplies | next-day | $ 9.00 |
| 17 | | | |

sales | shipping_rates

| | item_type | priority | shipping_cost |
|---|---|---|---|
| 0 | Baby Food | 2-day | 5 |
| 1 | Baby Food | next-day | 7 |
| 2 | Cereal | 2-day | 8 |
| 3 | Cereal | next-day | 11 |
| 4 | Fruit | 2-day | 5 |
| 5 | Fruit | next-day | 6 |
| 6 | Office Supplies | 2-day | 7 |
| 7 | Office Supplies | next-day | 9 |

nalyses

l data entry

# Data Analysis with pandas

Exploratory Analysis

## Essential inspection methods

```
# View first/last rows
df.head()        # First 5 rows (default)
df.head(10)      # First 10 rows
df.tail()        # Last 5 rows

# DataFrame dimensions
df.shape         # (rows, columns)

# Column names and types
df.columns       # Column names
df.dtypes        # Data type of each column

# Overall structure
df.info()        # Columns, dtypes, non-null counts, memory

# Statistical summary
df.describe()    # Count, mean, std, min, max, quartiles
```

## Quick visualization

```
# Histograms of all numeric columns
df.hist(figsize=(12, 8))

# Single column histogram
df['expression'].hist(bins=50)

# Value counts for categorical data
df['condition'].value_counts()

# Quick line plot
df.plot()

# Box plots by group
df.boxplot(column='value', by='group')

# Correlation matrix
df.corr()
```

## Typical exploration workflow:

1. Load data → 2. Check shape and columns → 3. Inspect dtypes → 4. Look for missing values → 5. Summary statistics → 6. Quick visualization

# Data Analysis with pandas

Data selection

## Selecting columns

```python
# Single column (returns Series)
expression = df['expression']

# Alternative (only for valid identifiers)
expression = df.expression

# Multiple columns (returns DataFrame)
subset = df[['gene_id', 'expression']]

# All columns except some
cols_to_keep = [c for c in df.columns if c != 'notes']
subset = df[cols_to_keep]

# Select by data type
numeric_cols = df.select_dtypes(include='number')
```

## Selecting rows

```python
# Boolean indexing (filtering)
treated = df[df['condition'] == 'treated']
high_expr = df[df['expression'] > 100]

# Multiple conditions with & (and) or | (or)
result = df[(df['expression'] > 100) &
            (df['condition'] == 'treated')]

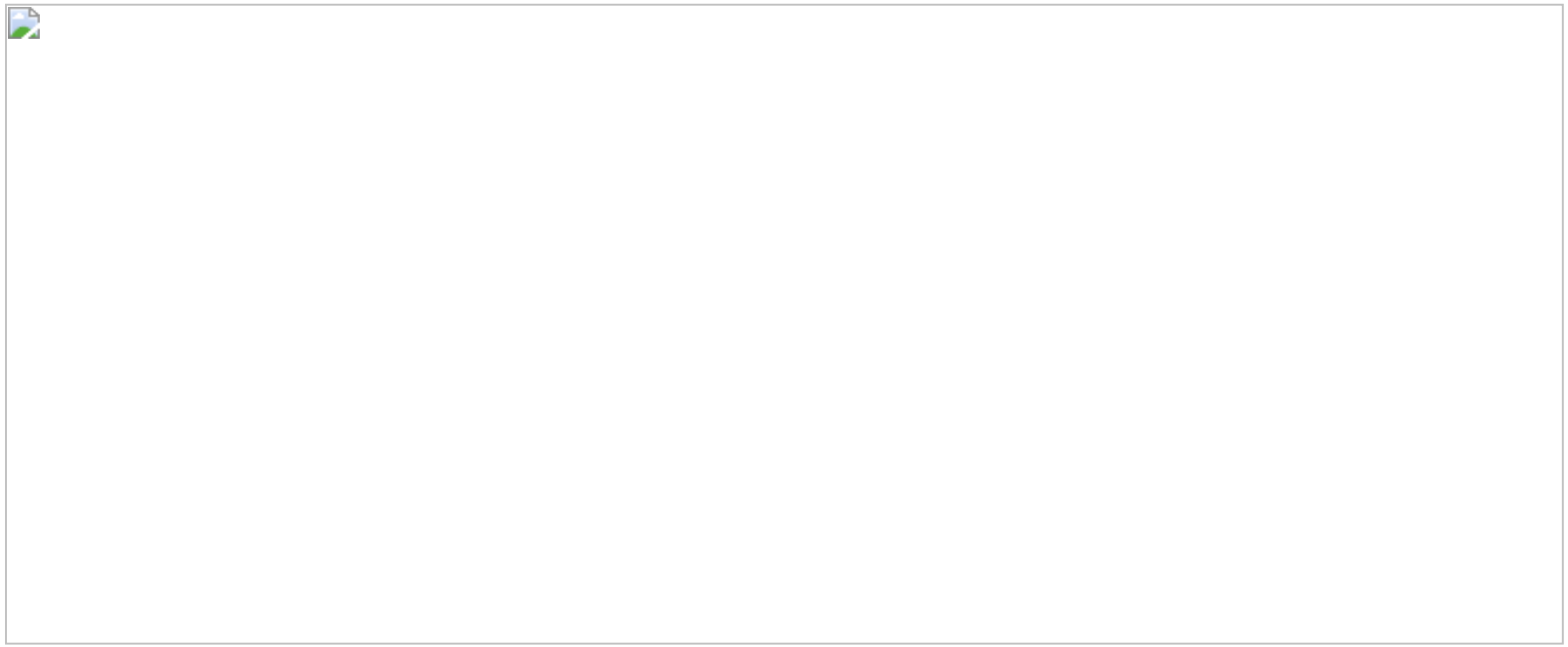
# Using query() method (cleaner syntax)
result = df.query('expression > 100 and condition == "treated"')

# Inverse selection with ~
not_treated = df[~(df['condition'] == 'treated')]
```

**Important:** Boolean indexing with multiple conditions requires parentheses around each condition and uses & (and) / | (or), not and/or

# Data Analysis with pandas

Data selection



**Important:** Boolean indexing with multiple conditions requires parentheses around each condition and uses `&` (and) / `|` (or), not and/or

**Use cases:** Selecting genes above threshold, filtering trials by condition, extracting cells from specific brain regions

# Data Analysis with pandas

Explicit indexing methods

## .loc[] - Label-based indexing

Access by row/column labels (names)

```
# Single value
value = df.loc[0, 'expressi

# Row slice (inclusive!)
subset = df.loc[0:5, 'expre

# Multiple rows and columns
subset = df.loc[[0, 2, 4],

# All rows, specific column
subset = df.loc[:, ['gene_i

# Boolean indexing with .lo
treated = df.loc[df['condit
```

## .iloc[] - Position-based indexing

Access by integer position (0-indexed)

```
# Single value (row 0, column 1)
                                    osition
                                    1]]
```

.loc[102] →
.iloc[3] →

| | CustomerID | Name | Age | Region | Income Strata | Sales |
|---|---|---|---|---|---|---|
| 101 | X1000 | John | 30 | North | High | 250 |
| 102 | X1010 | Ann | 19 | North | Medium | 5000 |
| 103 | X1020 | Joe | 25 | South | Medium | 132 |
| 104 | X1030 | Alice | 53 | East | Low | 400 |
| 105 | X1040 | Susan | 38 | South | Medium | 780 |
| 106 | X1050 | Bill | 68 | West | High | 223 |

**Key difference:** `.loc[0:5]` includes row 5 (inclusive), `.iloc[0:5]` excludes row 5 (exclusive like Python slicing)

**.loc[]** - When you know row/column names, or using boolean indexing

**.iloc[]** - When working with positions (first N rows, last column, etc.)

# Data Analysis with pandas

## Handling missing data: *detection*

### Detecting missing values

```
# Check for missing values (returns boolean DataFrame)
missing_mask = df.isna()  # or df.isnull()

# Check for non-missing values
present_mask = df.notna()  # or df.notnull()

# Count missing values per column
missing_count = df.isna().sum()

# Proportion of missing values
missing_prop = df.isna().sum() / len(df)

# Total missing values in DataFrame
total_missing = df.isna().sum().sum()

# Rows with any missing values
rows_with_na = df[df.isna().any(axis=1)]
```

**Common causes in experimental data:**

- Equipment failures or measurement errors
- Tracking loss in behavioral videos
- Below detection limit in omics experiments
- Incomplete trials or subject dropout
- Data entry errors or file corruption

**Important:** Always understand *why* data is missing before deciding how to handle it. The pattern of missingness can be scientifically meaningful. **Domain Knowledge Required!**

# Data Analysis with pandas

## Handling missing data: *deletion*

### Using .dropna()

```
# Drop rows with ANY missing value
df_clean = df.dropna()

# Drop rows where ALL values are missing
df_clean = df.dropna(how='all')

# Drop rows with missing in specific columns
df_clean = df.dropna(subset=['expression', 'condition'])

# Require minimum non-NA values
df_clean = df.dropna(thresh=5)  # Keep rows with ≥5 non-NA

# Drop columns with missing values
df_clean = df.dropna(axis=1)

# In-place modification
df.dropna(inplace=True)
```

### Good reasons to delete:

- Very few missing values (< 5%)
- Missing completely at random (MCAR)
- Plenty of remaining data
- Critical variables are missing

### Avoid deletion when:

- Large proportion missing (> 20%)
- Missing not at random (MNAR)
- Small dataset to begin with
- Pattern of missingness is informative

# Data Analysis with pandas

## Handling missing data: *imputation*

### Simple imputation

```python
# Fill with constant value
df_filled = df.fillna(0)
df['expression'] = df['expression'].fillna(0)

# Fill with mean/median/mode
df['expression'] = df['expression'].fillna(df['expression'].mean())
df['expression'] = df['expression'].fillna(df['expression'].median())

# Forward fill (propagate last valid value)
df_filled = df.fillna(method='ffill')

# Backward fill
df_filled = df.fillna(method='bfill')

# Limit number of fills
df_filled = df.fillna(method='ffill', limit=2)
```

### Interpolation for timeseries

```python
# Linear interpolation
df['position_x'] = df['position_x'].interpolate()

# Polynomial interpolation
df['trace'] = df['trace'].interpolate(method='polynomial',
                                      order=2)

# Time-based interpolation (for datetime index)
df_interpolated = df.interpolate(method='time')

# Interpolate only small gaps
df['signal'] = df['signal'].interpolate(limit=5)

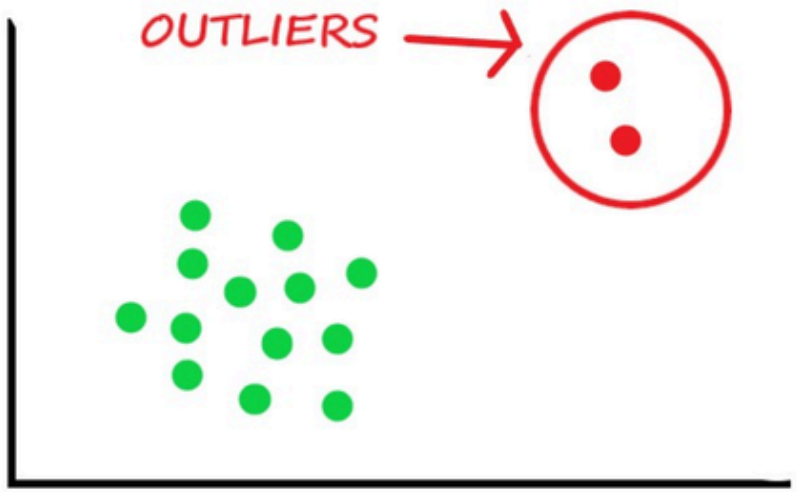
# Group-wise imputation
df['value'] = df.groupby('condition')['value'].transform(
    lambda x: x.fillna(x.mean()))
```

- **Mean/Median:** Numeric data, approximately normal distribution
- **Forward/Backward fill:** Sequential/temporal data (tracking, timeseries)
- **Interpolate:** Smooth continuous variables (position, fluorescence traces)
- **Zero:** When absence means true zero (gene not expressed, no event)
- **Group mean:** When groups differ systematically (condition, subject)

**Warning:** Imputation introduces assumptions and can affect statistical properties. Always report imputation methods in publications and consider sensitivity analyses.

# Data Analysis with pandas

## Detecting and Handling Outliers




```
# Box plot shows outliers automatically
df.boxplot(column='reaction_time', by='condition')

# Scatter plot for visual inspection
df.plot.scatter(x='trial_number', y='reaction_time')
```

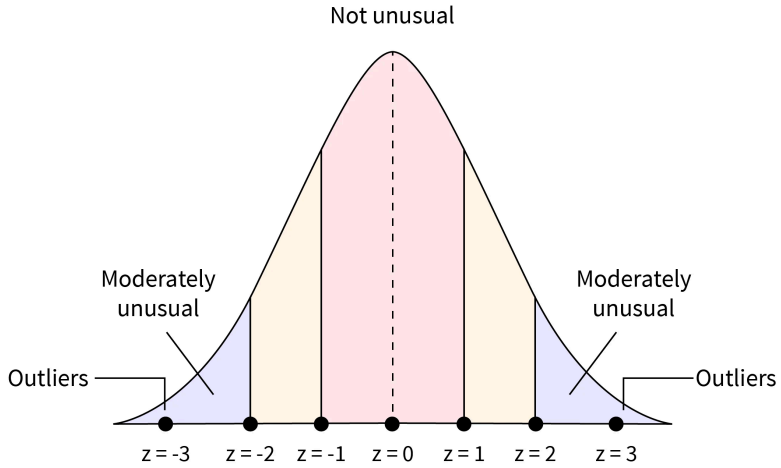
**Z-score method**: For approximately normal distributions

```
# Calculate z-scores
from scipy import stats
z_scores = np.abs(stats.zscore(df['reaction_time']))

# Flag outliers (|z| > 3)
outliers = z_scores > 3
print(f"Found {outliers.sum()} outliers")

# Remove outliers
df_clean = df[z_scores <= 3]

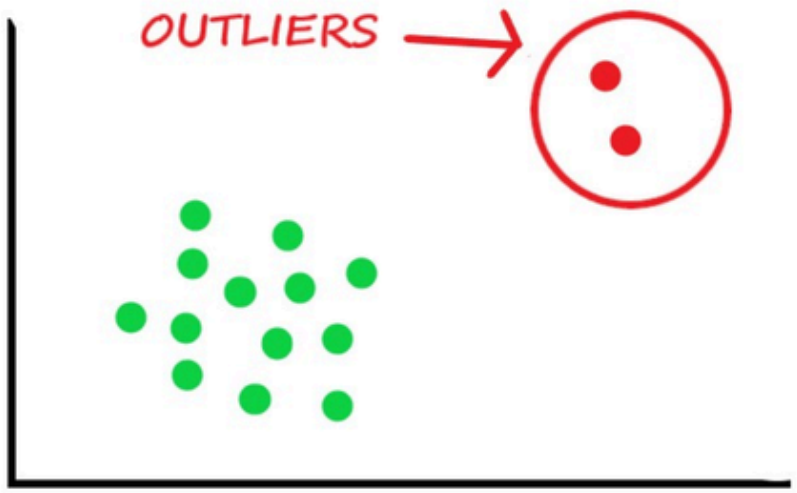
# Or replace with threshold
df.loc[z_scores > 3, 'reaction_time'] = df['reaction_time'].quantile(0.99)
```




**Critical decision:** Not all extreme values are errors! Outliers might be rare but real events (bursts, extreme behaviors). Always investigate before removing. **Domain Knowledge Required!**

# Data Analysis with pandas

## Detecting and Handling Outliers




```
# Box plot shows outliers automatically
df.boxplot(column='reaction_time', by='condition')

# Scatter plot for visual inspection
df.plot.scatter(x='trial_number', y='reaction_time')
```

**IQR method**: More robust to non-normal distributions

```
# Calculate IQR
Q1 = df['expression'].quantile(0.25)
Q3 = df['expression'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Flag outliers
outliers = (df['expression'] < lower_bound) | (df['expression'] > upper_bound)

# Remove outliers
df_clean = df[~outliers]
```



**Critical decision:** Not all extreme values are errors! Outliers might be rare but real events (bursts, extreme behaviors). Always investigate before removing. **Domain Knowledge Required!**

# Data Analysis with pandas

## Data Conversions

### Basic conversions

```
# Check current types
print(df.dtypes)

# Convert to integer
df['subject_id'] = df['subject_id'].astype(int)

# Convert to float
df['expression'] = df['expression'].astype(float)

# Convert to string
df['gene_id'] = df['gene_id'].astype(str)

# Convert to boolean
df['is_responsive'] = df['is_responsive'].astype(bool)

# Multiple columns at once
df = df.astype({'trial': int, 'correct': bool})
```

### Safe conversions

```
# Handle errors gracefully - convert invalid to NaN
df['value'] = pd.to_numeric(df['value'], errors='coerce')

# Or ignore errors (keep original)
df['value'] = pd.to_numeric(df['value'], errors='ignore')

# Convert to datetime
df['date'] = pd.to_datetime(df['date'])

# With custom format
df['date'] = pd.to_datetime(df['date'],
                            format='%Y-%m-%d')

# Handle errors in datetime
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

### Categorical data for memory efficiency

```
# Convert to category (saves memory for repeated values)
df['condition'] = df['condition'].astype('category')
df['brain_region'] = df['brain_region'].astype('category')

# Check memory savings
print(f"Memory usage: {df.memory_usage(deep=True)['condition'] / 1024:.2f} KB")
```

**Tip:** Use categorical type for columns with limited unique values (conditions, groups, labels). Can reduce memory by 90%+ for large datasets.

# Data Analysis with pandas



## Simple arithmetic operations

```
# Basic math operations
df['log_expression'] = np.log2(df['expression'] + 1)
df['normalized'] = (df['value'] - df['value'].mean()) / df['value'].std()

# Combining columns
df['fold_change'] = df['treated'] / df['control']
df['total'] = df['correct'] + df['incorrect']
df['accuracy'] = df['correct'] / df['total']

# Conditional values with np.where
df['high_expr'] = np.where(df['expression'] > 100, 'high', 'low')

# Multiple conditions
df['category'] = np.select(
    [df['value'] < 10, df['value'] < 50, df['value'] >= 50],
    ['low', 'medium', 'high']
)
```

# Data Analysis with pandas



## Using .apply() for custom functions

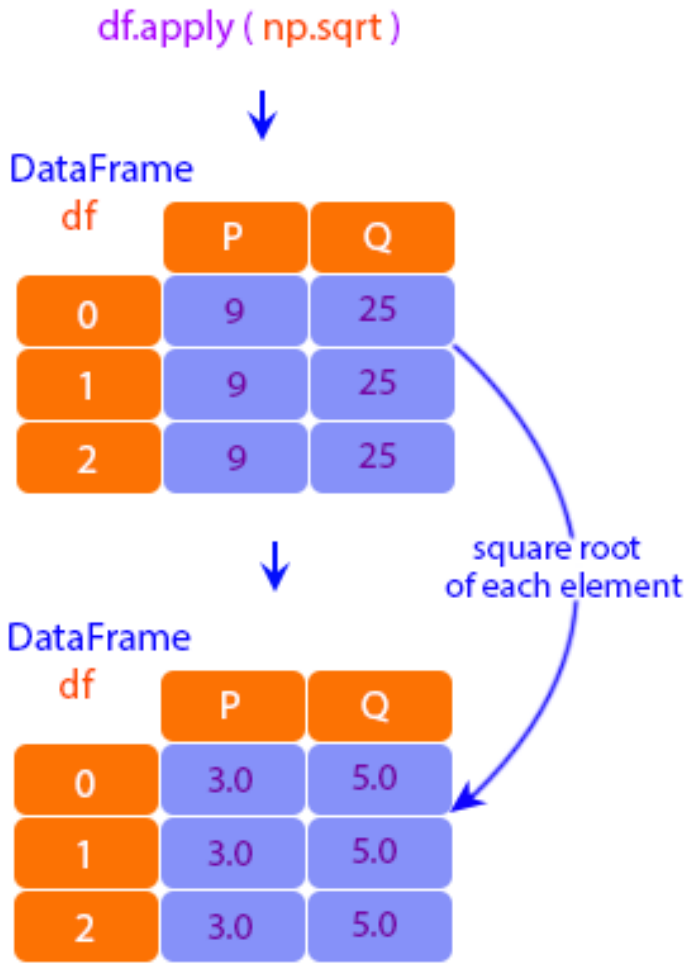
```python
# Apply function to single column
def categorize_expression(value):
    if value < 50:
        return 'low'
    elif value < 150:
        return 'medium'
    else:
        return 'high'

df['expr_category'] = df['expression'].apply(categorize_expression)

# Lambda functions for simple operations
df['doubled'] = df['value'].apply(lambda x: x * 2)

# Apply to multiple columns (row-wise)
df['sum'] = df[['col1', 'col2', 'col3']].apply(sum, axis=1)

# Map values from dictionary
mapping = {'A': 1, 'B': 2, 'C': 3}
df['numeric'] = df['letter'].map(mapping)
```




**Performance note:** Vectorized operations are much faster. Use `np.where()`, `np.select()`, or direct arithmetic when possible. Reserve .apply() for complex operations that can't be vectorized.

# Data Analysis with pandas

## Transforming Data: *GroupBy operations*

```
# Single grouping variable
grouped = df.groupby('condition')

# Common aggregations
df.groupby('condition').mean()
df.groupby('condition').sum()
df.groupby('condition').count()
df.groupby('condition').std()
df.groupby('condition').min()
df.groupby('condition').max()

# Aggregate specific column
df.groupby('condition')['reaction_time'].mean()

# Multiple grouping variables
df.groupby(['subject', 'condition']).mean()

# Example: Mean reaction time per condition
rt_by_condition = df.groupby('condition')['rt'].mean()
print(rt_by_condition)
# Output:
# condition
# control    0.523
# drug_A     0.412
# drug_B     0.398

# Access individual groups
for name, group in df.groupby('condition'):
    print(f"Condition: {name}")
    print(f"N trials: {len(group)}")
    print(f"Mean RT: {group['rt'].mean():.3f}\n")
```

**Understanding GroupBy object:**

- `df.groupby('col')` creates a GroupBy object (not a DataFrame)
- Must apply an aggregation to get results
- Can group by one or multiple columns
- Grouping columns become the index

**Common aggregations:**

- `.mean()`, `.median()` - Central tendency
- `.std()`, `.var()` - Variability
- `.sum()`, `.count()` - Totals
- `.min()`, `.max()` - Extremes
- `.first()`, `.last()` - Endpoints
- `.size()` - Group sizes (includes NaN)

**The GroupBy paradigm:** *Split* data into groups → *Apply* function to each group → *Combine* results

# Data Analysis with pandas

## Transforming Data: *Advanced aggregation*

```python
# Multiple aggregations with .agg()
df.groupby('condition').agg(['mean', 'std', 'count'])

# Different aggregations per column
df.groupby('condition').agg({
    'reaction_time': ['mean', 'std'],
    'accuracy': ['mean', 'sum'],
    'trial_id': 'count'
})

# Named aggregations (clearer column names)
df.groupby('condition').agg(
    mean_rt=('reaction_time', 'mean'),
    std_rt=('reaction_time', 'std'),
    n_trials=('trial_id', 'count'),
    total_correct=('correct', 'sum')
)

# Custom aggregation functions
def range_func(x):
    return x.max() - x.min()

df.groupby('condition').agg({
    'value': ['mean', range_func, 'count']
})

# Transform: keep original DataFrame shape
# (broadcasts group statistics back to all rows)
df['condition_mean'] = df.groupby('condition')['value'].transform('mean')
df['z_score'] = (df['value'] - df['condition_mean']) / \
                df.groupby('condition')['value'].transform('std')

# Filter groups by condition
# Keep only groups with > 10 observations
df_filtered = df.groupby('subject').filter(lambda x: len(x) > 10)
```

**.agg() vs .transform() vs .filter():**

- **.agg()**: Reduces groups to summary statistics (returns smaller DataFrame)
- **.transform()**: Broadcasts group statistics back to original rows (same shape as input)
- **.filter()**: Keeps/removes entire groups based on criteria

**Custom functions:**

- Can use any function that takes array, returns scalar
- Lambda functions for simple operations
- Named functions for complex logic
- Examples: range, CV, SEM, percentiles

# Data Analysis with pandas

## Transforming Data: *Advanced aggregation*

```python
# Multiple aggregations with .agg()
df.groupby('condition').agg(['mean', 'std', 'count'])

# Different aggregations per column
df.groupby('condition').agg({
    'reaction_time': ['mean', 'std'],
    'accuracy': ['mean', 'sum'],
    'trial_id': 'count'
})

# Named aggregations (clearer column names)
df.groupby('condition').agg(
    mean_rt=('reaction_time', 'mean'),
    std_rt=('reaction_time', 'std'),
    n_trials=('trial_id', 'count'),
    total_correct=('correct', 'sum')
)

# Custom aggregation functions
def range_func(x):
    return x.max() - x.min()

df.groupby('condition').agg({
    'value': ['mean', range_func, 'count']
})

# Transform: keep original DataFrame shape
# (broadcasts group statistics back to all rows)
df['condition_mean'] = df.groupby('condition')['value'].transform('mean')
df['z_score'] = (df['value'] - df['condition_mean']) / \
                df.groupby('condition')['value'].transform('std')

# Filter groups by condition
# Keep only groups with > 10 observations
df_filtered = df.groupby('subject').filter(lambda x: len(x) > 10)
```

**Practical examples:**

- **Z-score per condition**: Use transform to get group mean/std, calculate z-scores
- **Filter low-trial subjects**: Remove subjects with < N trials
- **Comprehensive summary**: Mean, SEM, N for each group
- **Normalize to control**: Transform with group-specific baselines

# Data Analysis with pandas

```
# Basic pivot table
pivot = pd.pivot_table
    df,
    values='reaction_t
    index='subject',
    columns='condition
    aggfunc='mean'
)
# Result: subjects as

# Multiple aggregation
pivot_multi = pd.pivot_
    df,
    values='reaction_t
    index='subject',
    columns='condition
    aggfunc=['mean', '
)

# Multiple values
pivot_multi_vals = pd.
    df,
    values=['reaction_
    index='subject',
    columns='condition',
    aggfunc='mean'
)
```

df

| | foo | bar | baz | zoo |
|---|---|---|---|---|
| 0 | one | A | 1 | x |
| 1 | one | B | 2 | y |
| 2 | one | C | 3 | z |
| 3 | two | A | 4 | q |
| 4 | two | B | 5 | w |
| 5 | two | C | 6 | t |

Stacked

## Transforming Data: *Pivot Tables*

```
df.pivot(index='foo',
         columns='bar',
         values='baz')
```

| bar | A | B | C |
|---|---|---|---|
| foo | | | |
| one | 1 | 2 | 3 |
| two | 4 | 5 | 6 |

Record

**Pivot tables:** Create cross-tabulated summaries with one variable as rows, another as columns

https://pandas.pydata.org/docs/user_guide/reshaping.html#

# Data Analysis with pandas

## Transforming Data: *Long vs Wide format*

```
# Wide to Long: pd.melt() (unpivot)
df_long = pd.melt(
    df_wide,
    id_vars=['s
    value_vars=
    var_name='t
    value_name=
)

# Long to Wide:
df_wide = df_lo
    index='subj
    columns='ti
    values='sco
)

# Multiple valu
df_long_multi =
    df,
    id_vars=['s
    value_vars=
    var_name='g
    value_name=
)
```

df3

| | first | last | height | weight |
|---|---|---|---|---|
| 0 | John | Doe | 5.5 | 130 |
| 1 | Mary | Bo | 6.0 | 150 |



`df3.melt(id_vars=['first', 'last'])`

| | first | last | variable | value |
|---|---|---|---|---|
| 0 | John | Doe | height | 5.5 |
| 1 | Mary | Bo | height | 6.0 |
| 2 | John | Doe | weight | 130 |
| 3 | Mary | Bo | weight | 150 |

**Wide format:** Each variable has its own column (e.g., pre_score, post_score) - compact but harder to analyze

**Long format (tidy):** One observation per row, variables in separate columns (e.g., timepoint, score) - easier for analysis/plotting

# Data Analysis with pandas

## Transforming Data: *concatenate*

```python
# Vertical concatenation (stack rows, default)
df_combined = pd.concat([df1, df2, df3], axis=0)
# Columns must match (or use ignore_index=True)

# Reset index after concatenation
df_combined = pd.concat([df1, df2], ignore_index=True)

# Horizontal concatenation (stack columns)
df_wide = pd.concat([df1, df2], axis=1)
# Rows must match (aligned by index)

# Add identifying key for source
df_combined = pd.concat(
    [df1, df2, df3],
    keys=['session1', 'session2', 'session3']
)
# Creates multi-level index

# Or add as column
df1['session'] = 'session1'
df2['session'] = 'session2'
df_combined = pd.concat([df1, df2])

# Example: Combine multiple subject files
dfs = []
for file in ['subj01.csv', 'subj02.csv', 'subj03.csv']:
    df = pd.read_csv(file)
    df['subject'] = file.replace('.csv', '')
    dfs.append(df)
all_data = pd.concat(dfs, ignore_index=True)
```

**Key parameters:**

- `axis=0`: Vertical (rows, default)
- `axis=1`: Horizontal (columns)
- `ignore_index=True`: Reset row numbers
- `keys=['name1', 'name2']`: Add source identifier
- `join='outer'/'inner'`: How to handle mismatched columns

**Axis 0 vs Axis 1:**

- **axis=0** (vertical): Stack DataFrames top-to-bottom (more rows)
- **axis=1** (horizontal): Stack DataFrames side-by-side (more columns)
- Think: axis is the direction you're stacking *along*

**Concatenation:** Combine multiple DataFrames by stacking them - simple and fast for same-structure data

# Data Analysis with pandas

## Transforming Data: *join & merge*

```python
# Inner join: keep only matching rows (default)
merged = pd.merge(df_trials, df_subjects, on='subject_id')
# Only subjects present in BOTH DataFrames

# Left join: keep all left, match from right
merged = pd.merge(df_trials, df_subjects,
                  on='subject_id', how='left')
# All trials kept, subject info added (NaN if no match)

# Right join: keep all right, match from left
merged = pd.merge(df_trials, df_subjects,
                  on='subject_id', how='right')
# All subjects kept, trial info added (NaN if no match)

# Outer join: keep all from both
merged = pd.merge(df_trials, df_subjects,
                  on='subject_id', how='outer')
# Everything kept, NaN where no match

# Merge on multiple keys
merged = pd.merge(df1, df2,
                  on=['subject_id', 'session'])

# Different column names
merged = pd.merge(df1, df2,
                  left_on='subject', right_on='participant_id')

# Merge on index instead
merged = pd.merge(df1, df2, left_index=True, right_index=True)
```

**Join types (how parameter):**

- **inner** (default): Only matching keys from both
- **left**: All from left, matching from right
- **right**: All from right, matching from left
- **outer**: All from both (union)



**Key parameters:**

- `on`: Column name(s) to merge on (same in both)
- `left_on, right_on`: Different column names
- `left_index, right_index`: Merge on index
- `how`: Type of join (inner/left/right/outer)
- `suffixes`: Rename duplicate columns
- `validate`: Check relationship (one_to_one, one_to_many, etc.)

**Merging:** Combine DataFrames based on shared key column(s) - like SQL joins, for relating different datasets

# Data Analysis with pandas + scipy

Statistics and signal processing

**SciPy** - Collection of mathematical algorithms and convenience functions

- Built on NumPy arrays
- Provides efficient, well-tested routines
- C/Fortran extensions for speed
- Actively maintained and extensible

*"NumPy provides the data structures, SciPy provides the application layer."*

**Today's focus:** scipy.stats for statistical analysis and scipy.signal for filtering and peak detection

## Key subpackages for data analysis

| Package | Purpose |
|---|---|
| **scipy.stats** | Statistical functions and distributions |
| **scipy.signal** | Signal processing (filtering, peaks) |
| **scipy.optimize** | Optimization and curve fitting |
| **scipy.integrate** | Numerical integration |
| **scipy.interpolate** | Interpolation and smoothing |
| **scipy.linalg** | Linear algebra routines |

```
# Common import patterns
import scipy
from scipy import stats, signal
import scipy.stats as st

# Check version
print(scipy.__version__)
```

# Probability & Statistics

## Descriptive Statistics

### Basic statistics with pandas

```python
# Basic descriptive statistics
print(df['expression'].mean())
print(df['expression'].median())
print(df['expression'].std())
print(df['expression'].var())
print(df['expression'].min())
print(df['expression'].max())

# All at once
print(df['expression'].describe())

# By group
summary = df.groupby('condition')['expression'].describe()

# Custom percentiles
df['expression'].quantile([0.25, 0.5, 0.75, 0.95])
```

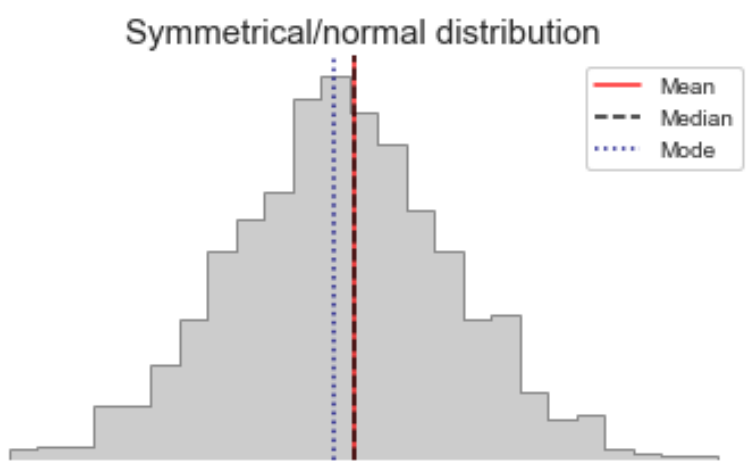
### Extended statistics with scipy.stats

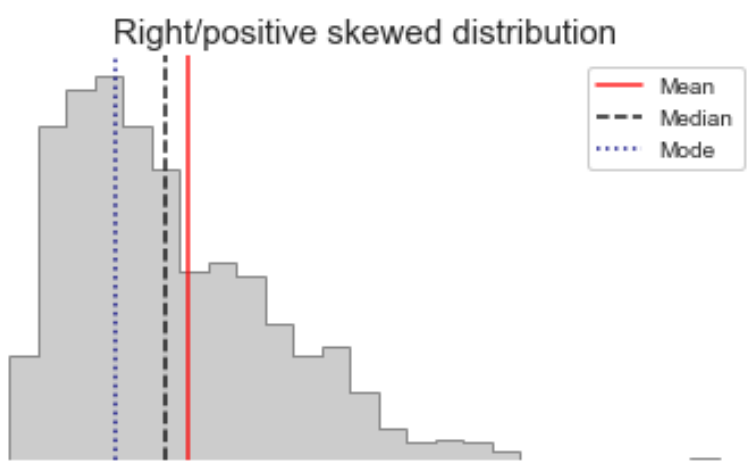
```python
from scipy import stats

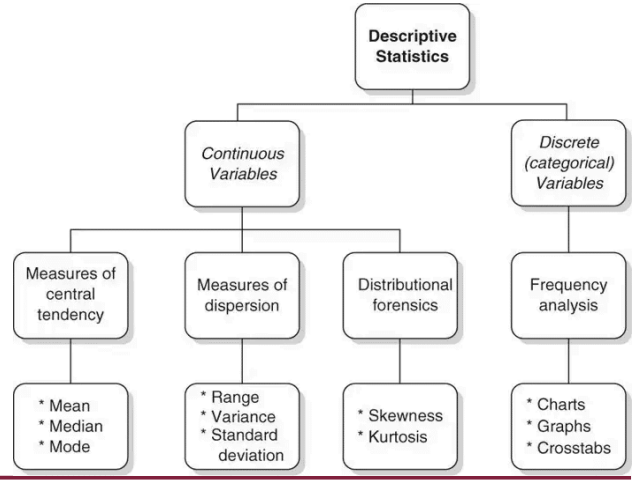
# Comprehensive description
n, (min_val, max_val), mean, var, skew, kurt = stats.describe(data)

print(f"N: {n}")
print(f"Mean: {mean:.2f}")
print(f"Variance: {var:.2f}")
print(f"Skewness: {skew:.2f}")  # Symmetry
print(f"Kurtosis: {kurt:.2f}")  # Tail heaviness

# Individual measures
stats.skew(data)      # Distribution asymmetry
stats.kurtosis(data)  # Tail behavior
stats.sem(data)       # Standard error of mean
```



# Probability & Statistics

Probability distributions

## Creating distributions

```python
from scipy.stats import norm, expon, lognorm
import numpy as np

# Normal (Gaussian) distribution
dist = norm(loc=100, scale=15)  # mean=100, std=15

# Generate random samples
samples = dist.rvs(size=1000)

# Calculate probabilities
x = np.linspace(50, 150, 100)
pdf = dist.pdf(x)  # Probability density
cdf = dist.cdf(x)  # Cumulative probability

# Percent point function (inverse CDF)
ppf = dist.ppf(0.95)  # Value at 95th percentile
```

## Distribution parameters

```python
# All distributions have loc and scale
# loc: shifts distribution left/right
# scale: stretches/compresses

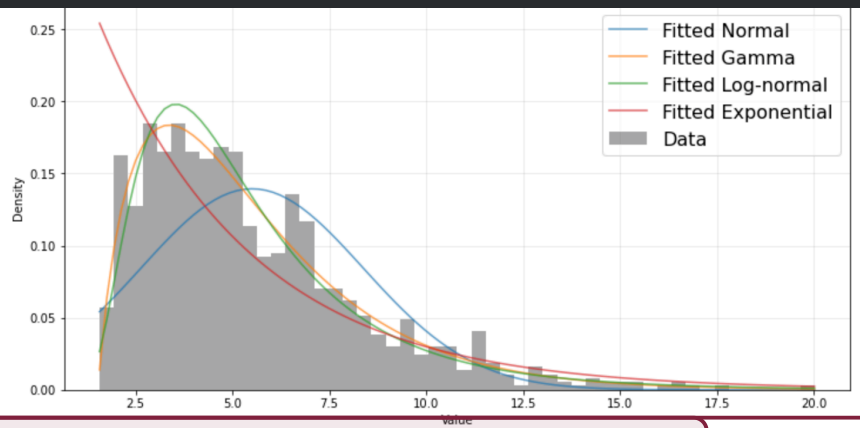
# Normal distribution
norm(loc=0, scale=1)      # Standard normal
norm(loc=10, scale=2)     # Mean=10, std=2

# Exponential distribution
expon(scale=5)            # scale = 1/lambda

# Log-normal distribution
lognorm(s=0.5, loc=0, scale=1)  # s=shape parameter

# Get theoretical statistics
mean, var = dist.stats(moments='mv')
print(f"Mean: {mean}, Variance: {var}")
```

- **Normal:** Measurement errors, many biological variables
- **Log-normal:** Gene expression levels, synaptic weights
- **Exponential:** Inter-spike intervals, waiting times
- **Poisson:** Spike counts in fixed time windows
- **Gamma:** Reaction times, neural response latencies




**Why use distributions?** Model theoretical expectations, generate synthetic data for testing, calculate probabilities, and validate assumptions for statistical tests.

# Probability & Statistics

## Fitting distributions to data

### Parameter estimation with .fit()

```python
from scipy.stats import norm, expon, lognorm

# Fit normal distribution to data
data = df['reaction_time'].dropna()
mu, sigma = norm.fit(data)
print(f"Mean: {mu:.2f}, Std: {sigma:.2f}")

# Fit exponential distribution
scale = expon.fit(data, floc=0)[1]  # Fix location at 0

# Fit log-normal distribution
shape, loc, scale = lognorm.fit(data, floc=0)

# Compare fitted distribution to data
x = np.linspace(data.min(), data.max(), 100)
plt.hist(data, bins=30, density=True, alpha=0.5)
plt.plot(x, norm.pdf(x, mu, sigma), 'r-', label='Fitted')
```

### Kernel Density Estimation

Non-parametric density estimation

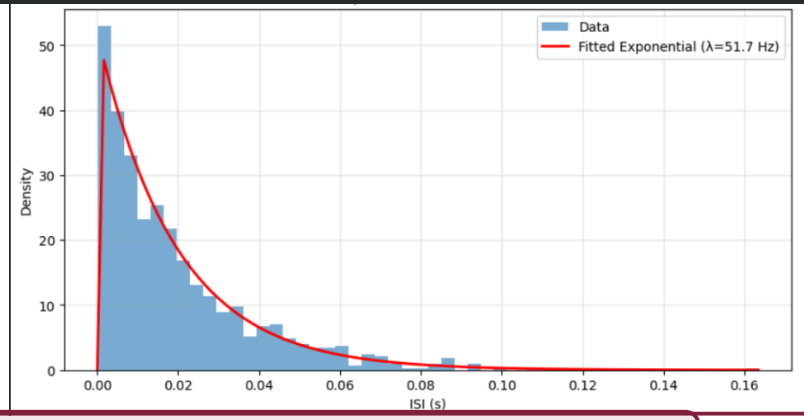
```python
from scipy.stats import gaussian_kde

# Create KDE estimator
kde = gaussian_kde(data)

# Evaluate density at points
x = np.linspace(data.min(), data.max(), 200)
density = kde(x)

# Plot smoothed density estimate
plt.plot(x, density, label='KDE')
plt.hist(data, bins=30, density=True, alpha=0.3)
```

- Characterize the underlying process generating your data
- Test if data follows expected theoretical distribution
- Generate synthetic data matching real data properties
- Calculate probabilities and percentiles
- Compare distributions between experimental conditions




**Example:** Fit exponential distribution to inter-spike intervals to estimate neuronal firing rate parameter, or fit log-normal to gene expression to model biological variability.

# Probability & Statistics

## Fitting distributions to data

```python
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt

# Example: Interspike intervals (ISI)
isi_data = np.array([0.015, 0.023, 0.018, 0.031,
                     0.012, 0.025, 0.019, ...])

# Fit exponential distribution
params = stats.expon.fit(isi_data)
loc, scale = params

# Fitted distribution
fitted_dist = stats.expon(loc=loc, scale=scale)

# Generate theoretical values
x = np.linspace(0, isi_data.max(), 100)
pdf_fitted = fitted_dist.pdf(x)

# Visualize fit
plt.hist(isi_data, bins=30, density=True,
         alpha=0.6, label='Data')
plt.plot(x, pdf_fitted, 'r-',
         label=f'Exponential fit (λ={1/scale:.2f})')
plt.xlabel('ISI (s)')
plt.ylabel('Density')
plt.legend()
```

**Common distributions in neuroscience:**

- **Normal** (`stats.norm`): Measurement errors, population averages
- **Exponential** (`stats.expon`): Interspike intervals, waiting times
- **Poisson** (`stats.poisson`): Spike counts in fixed intervals
- **Log-normal** (`stats.lognorm`): Synaptic weights, expression levels
- **Gamma** (`stats.gamma`): Reaction times, neural latencies
- **Beta** (`stats.beta`): Proportions, probabilities

**Goodness of fit tests:**

- `stats.kstest()` - Kolmogorov-Smirnov test
- `stats.chisquare()` - Chi-square test
- Visual: Q-Q plot with `stats.probplot()`

# Probability & Statistics

## Hypothesis testing: *one-sample*

- **$H_0$ (null hypothesis):** No effect, no difference
- **$H_1$ (alternative hypothesis):** There is an effect or difference
- **p-value:** Probability of observing data this extreme if $H_0$ is true
- **Significance level (α):** Threshold for rejecting $H_0$ (commonly 0.05)
- **Decision:** If $p < \alpha$, reject $H_0$; otherwise, fail to reject $H_0$

### One-sample t-test

Does sample mean differ from expected

```python
from scipy.stats import ttest_1samp

# Test if mean differs from theoretical value
data = df['expression'].dropna()
statistic, pvalue = ttest_1samp(data, popmean=100)

print(f"t-statistic: {statistic:.3f}")
print(f"p-value: {pvalue:.4f}")

if pvalue < 0.05:
    print("Reject H₀: Mean differs from 100")
else:
    print("Fail to reject H₀")
```

### Kolmogorov-Smirnov test

Does sample follow expected distribution?

```python
from scipy.stats import kstest

# Test if data follows normal distribution
# Fit parameters first
mu, sigma = norm.fit(data)
statistic, pvalue = kstest(data, 'norm', args=(mu, sigma))

print(f"KS statistic: {statistic:.3f}")
print(f"p-value: {pvalue:.4f}")

if pvalue < 0.05:
    print("Reject H₀: Data not normally distributed")
else:
    print("Data consistent with normal distribution")
```

**Use cases:** Test if baseline firing rate differs from zero, verify normality before parametric tests, check if data match theoretical expectations.

# Probability & Statistics

## Hypothesis testing: *two-sample*

### Independent samples t-test

Do two groups have different means?

```python
from scipy.stats import ttest_ind

# Compare two independent groups
control = df[df['condition'] == 'control']['expression']
treated = df[df['condition'] == 'treated']['expression']

statistic, pvalue = ttest_ind(control, treated)

print(f"Control mean: {control.mean():.2f}")
print(f"Treated mean: {treated.mean():.2f}")
print(f"t-statistic: {statistic:.3f}")
print(f"p-value: {pvalue:.4f}")

if pvalue < 0.05:
    print("Groups differ significantly")
```

### Non-parametric alternative

When normality assumption violated

```python
from scipy.stats import mannwhitneyu

# Mann-Whitney U test (non-parametric)
# No assumption of normality required
statistic, pvalue = mannwhitneyu(control, treated)

print(f"U-statistic: {statistic:.3f}")
print(f"p-value: {pvalue:.4f}")

# Two-sample KS test (compare distributions)
from scipy.stats import ks_2samp
statistic, pvalue = ks_2samp(control, treated)

print(f"KS statistic: {statistic:.3f}")
print(f"p-value: {pvalue:.4f}")
```

| Situation | Test | Function |
|---|---|---|
| Normal distributions, equal variance | Independent t-test | `ttest_ind()` |
| Non-normal or ordinal data | Mann-Whitney U | `mannwhitneyu()` |
| Compare full distributions | Two-sample KS test | `ks_2samp()` |

**Common use case:** Compare gene expression between control and knockout mice, firing rates before/after drug application, or behavioral performance between experimental groups.

# Probability & Statistics

## Correlation analysis: *Pearson*

**Pearson correlation coefficient (r):** Measures *linear* relationship between two continuous variables

- Range: -1 (perfect negative) to +1 (perfect positive), 0 = no linear relationship
- **Assumptions:** Linear relationship, continuous variables, approximately normal distributions, no outliers

```python
from scipy import stats
import pandas as pd

# Single pair correlation
r, p = stats.pearsonr(df['gene_A'], df['gene_B'])
print(f"r = {r:.3f}, p = {p:.4f}")

# Pairwise correlation matrix (pandas)
corr_matrix = df[['gene_A', 'gene_B',
                  'gene_C']].corr(method='pearson')
print(corr_matrix)

# Example: Gene expression correlation
genes = df[['BDNF', 'TrkB', 'CREB', 'Arc']]
corr_matrix = genes.corr()

# Find highly correlated pairs (|r| > 0.7)
high_corr = corr_matrix[(corr_matrix.abs() > 0.7)
                        & (corr_matrix < 1.0)]
```

**Interpreting correlation strength:**

- $|r| < 0.3$: Weak correlation
- $0.3 \leq |r| < 0.7$: Moderate correlation
- $|r| \geq 0.7$: Strong correlation

⚠️ **Critical caveats:**

- **Correlation ≠ causation**
- Pearson only captures *linear* relationships
- Outliers can heavily influence r
- Statistical significance (p-value) depends on sample size
- Small p-value doesn't mean strong/meaningful relationship

# Probability & Statistics

## Correlation analysis: *Spearman & Kendall*

```python
from scipy import stats

# Spearman's rank correlation
rho, p = stats.spearmanr(df['dose'],
                         df['response'])
print(f"Spearman ρ = {rho:.3f}, p = {p:.4f}")

# Kendall's tau
tau, p = stats.kendalltau(df['dose'],
                          df['response'])
print(f"Kendall τ = {tau:.3f}, p = {p:.4f}")

# Compare all three methods
pearson_r, _ = stats.pearsonr(x, y)
spearman_rho, _ = stats.spearmanr(x, y)
kendall_tau, _ = stats.kendalltau(x, y)

print(f"Pearson:  {pearson_r:.3f}")
print(f"Spearman: {spearman_rho:.3f}")
print(f"Kendall:  {kendall_tau:.3f}")

# Pandas correlation matrix with Spearman
corr_spearman = df.corr(method='spearman')

# Example: Ordinal behavioral scores
# Pain scale (1-10) vs grimace score (0-5)
rho, p = stats.spearmanr(df['pain_scale'],
                         df['grimace_score'])
```

### Spearman's ρ (rho):

- Pearson correlation on *ranks* instead of raw values
- Captures **monotonic** relationships (not just linear)
- Robust to outliers
- Works with ordinal data
- Range: -1 to +1 (like Pearson)

### Kendall's τ (tau):

- Based on concordant/discordant pairs
- More robust to tied ranks than Spearman
- Better for small samples (n < 30)
- Range: -1 to +1
- More computationally intensive

**When Pearson isn't appropriate:** Non-linear but monotonic relationships, ordinal data, outliers present, non-normal distributions

# Signal Processing

Examples

**Prote**

Mass
signa

**Pe**
nc

**Ba**
ar

**Si**
ac

**De**
sig

**Spectral Alignment:** Time-warping algorithms to align retention times

tures

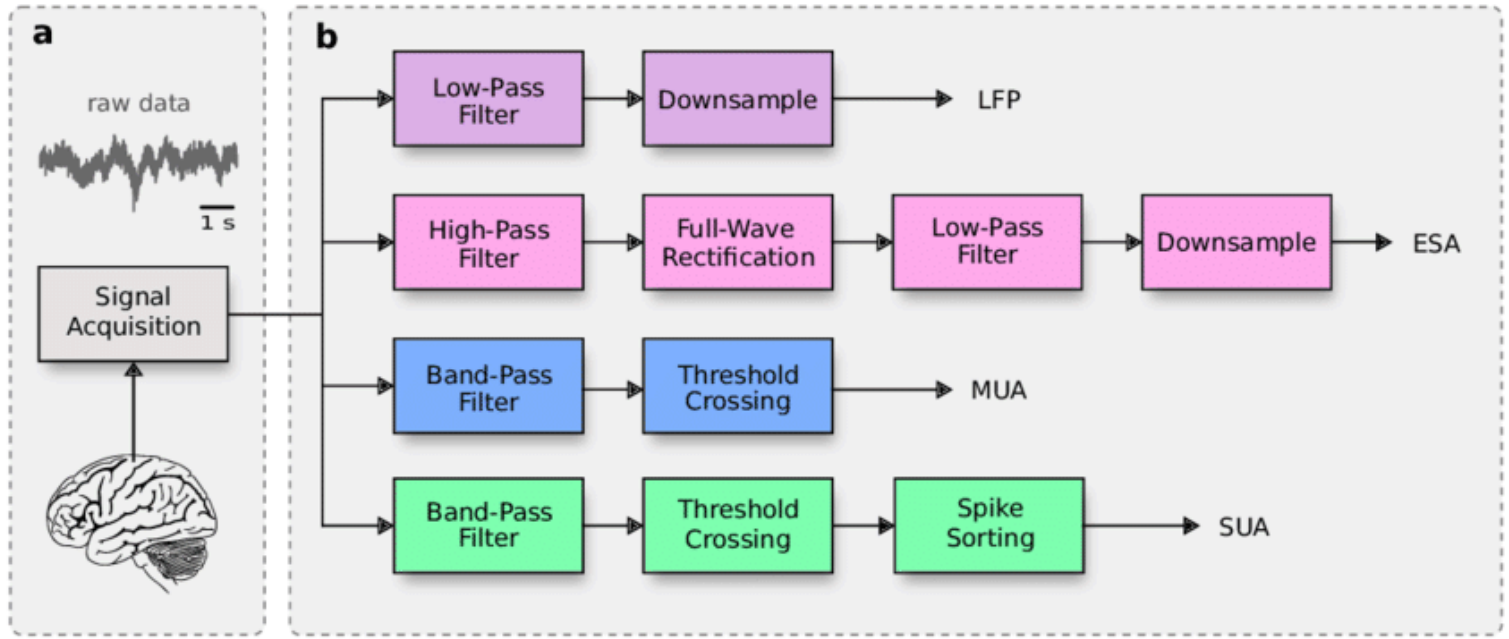
phase errors

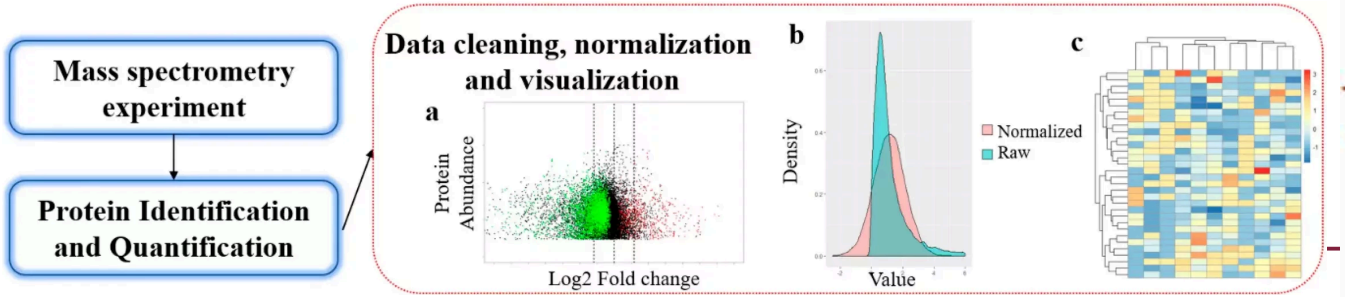
tabolite

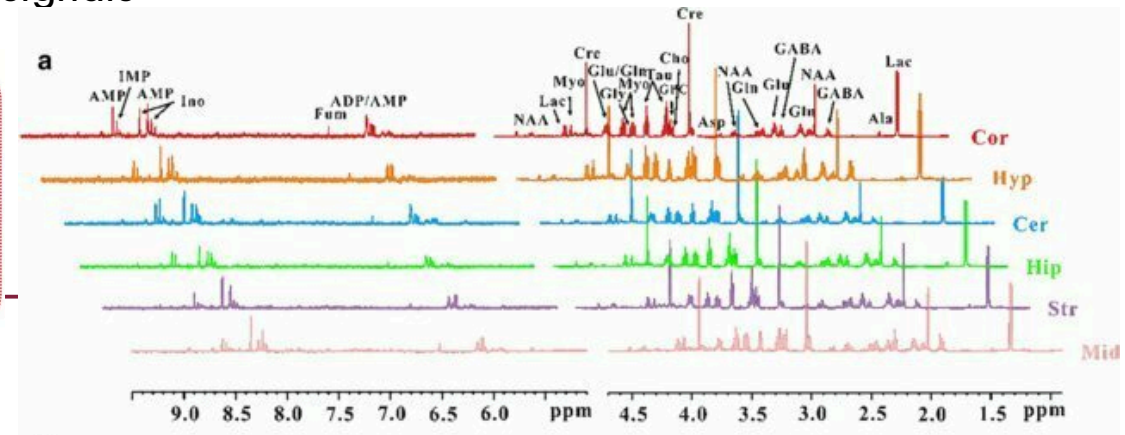
duce

hift

**Deconvolution:** Separating overlapping metabolite signals



# Signal Processing

## Filtering

```python
from scipy import signal
import numpy as np

# Sample rate and signal
fs = 1000  # Hz
t = np.linspace(0, 1, fs)
noisy_signal = np.sin(2*np.pi*10*t) + 0.5*np.random.randn(fs)

# 1. Low-pass filter (remove high-frequency noise)
# Keep frequencies below cutoff
cutoff = 20  # Hz
nyquist = fs / 2
normalized_cutoff = cutoff / nyquist
b, a = signal.butter(4, normalized_cutoff,
                     btype='low')
filtered = signal.filtfilt(b, a, noisy_signal)

# 2. High-pass filter (remove baseline drift)
# Keep frequencies above cutoff
cutoff = 0.5  # Hz
b, a = signal.butter(4, cutoff/nyquist,
                     btype='high')
filtered = signal.filtfilt(b, a, noisy_signal)

# 3. Band-pass filter (isolate frequency band)
# Keep frequencies between low and high
low, high = 8, 12  # Hz (alpha band for EEG)
b, a = signal.butter(4, [low/nyquist, high/nyquist],
                     btype='band')
filtered = signal.filtfilt(b, a, noisy_signal)
```

**Filter types:**

- **Low-pass**: Keep low frequencies (smoothing, noise removal)
- **High-pass**: Keep high frequencies (remove drift/DC offset)
- **Band-pass**: Keep middle frequencies (isolate oscillations)
- **Band-stop/Notch**: Remove specific frequency (e.g., 50/60 Hz line noise)

**Key parameters:**

- **Order**: Filter steepness (higher = sharper cutoff, more ringing)
- **Cutoff frequency**: Transition point (normalize by Nyquist)
- `filtfilt()`: Zero-phase filtering (forward + backward pass)

⚠️ **Important:**

- Always use `filtfilt()` to avoid phase distortion
- Normalize cutoff by Nyquist frequency (fs/2)
- Higher order = sharper but more edge artifacts

**Why filter signals?** Remove noise, isolate frequency bands of interest, remove baseline drift, smooth data

# Signal Processing

## Smoothing & Denoising

```python
from scipy import signal, ndimage
import numpy as np
import pandas as pd

# 1. Moving average (simple, fast)
window = 5
smoothed = np.convolve(noisy_signal, np.ones(window)/window, mode='same')

# Or with pandas (handles edges better)
df['smoothed'] = df['signal'].rolling(window=5, center=True).mean()

# 2. Savitzky-Golay filter (preserves peaks)
# Fits polynomial to local windows
smoothed = signal.savgol_filter(
    noisy_signal,
    window_length=11,  # Must be odd
    polyorder=3        # Polynomial degree
)

# 3. Gaussian smoothing
sigma = 2  # Standard deviation
smoothed = ndimage.gaussian_filter1d(
    noisy_signal, sigma)

# 4. Median filter (removes spikes/outliers)
smoothed = signal.medfilt(noisy_signal,
                          kernel_size=5)

# Example: Smooth calcium imaging trace
ca_trace = df['fluorescence'].values
smoothed_ca = signal.savgol_filter(
    ca_trace, window_length=21, polyorder=2)
df['ca_smoothed'] = smoothed_ca
```

### Method comparison:

| | | |
|---|---|---|
| **Moving avg** | General smoothing | Blunts peaks |
| **Savitzky-Golay** | Preserving peaks | Sensitive to outliers |
| **Gaussian** | Uniform smoothing | Smooths edges |
| **Median** | Spike removal | Can distort shape |

### Choosing window size:

- Larger window = more smoothing, more lag
- Should be smaller than features of interest
- Savitzky-Golay: window must be odd number
- Rule of thumb: start with ~5-10% of data length

**Smoothing vs Filtering:** Smoothing reduces noise while preserving overall trends; simpler than frequency-domain filtering

# Signal Processing

```python
from scipy import signal
import numpy as np

# Basic peak detection
peaks, properties = signal.find_peaks(
    signal_data,
    height=0.5,       # Minimum peak height
    distance=20,      # Min distance between peaks
    prominence=0.3,   # How much peak stands out
    width=5           # Minimum peak width
)

# Get peak properties
peak_heights = properties['peak_heights']
peak_widths = properties['widths']
peak_prominences = properties['prominences']


# Example: Calcium imaging transients
ca_trace = df['fluorescence'].values
peaks, props = signal.find_peaks(
    ca_trace,
    height=np.median(ca_trace) + 2*np.std(ca_trace),
    distance=int(0.5 * fps),  # Min 0.5s between
    prominence=0.2
)

# Calculate event frequency
duration = len(ca_trace) / fps  # seconds
event_frequency = len(peaks) / duration  # Hz

# Extract peak amplitudes
amplitudes = ca_trace[peaks]
mean_amplitude = np.mean(amplitudes)
```

## Peak Detection

**Key parameters:**

- **height**: Minimum peak height (absolute value or tuple for range)
- **distance**: Minimum samples between peaks (prevents duplicates)
- **prominence**: How much peak stands out from baseline
- **width**: Minimum peak width in samples
- **threshold**: Minimum vertical distance to neighbors

**Returned properties:**

- `peak_heights`: Y-values at peaks
- `prominences`: How much peak stands out
- `widths`: Peak widths at half-prominence
- `left_bases`, `right_bases`: Peak boundaries

**Parameter tuning strategy:**

1. Start with only `height` threshold
2. Add `distance` if getting duplicate detections
3. Add `prominence` to filter small bumps
4. Add `width` for temporal constraints
5. Visualize and iterate!

**Why detect peaks?** Identify events (calcium transients, action potentials, behavioral events), quantify event properties, calculate frequencies

# Signal Processing

```python
import pandas as pd
import numpy as np

# Convert strings to datetime
df['timestamp'] = pd.to_datetime(df['timestamp'])

# Common formats
pd.to_datetime('2024-01-15')
pd.to_datetime('15/01/2024', format='%d/%m/%Y')
pd.to_datetime('2024-01-15 14:30:00')

# Set as index for time-aware operations
df = df.set_index('timestamp')

# Time-based slicing (strings work!)
df['2024-01-15']              # Single day
df['2024-01']                  # Entire month
df['2024-01-15':'2024-01-20']  # Date range

# Extract time components
df['hour'] = df.index.hour
df['day_of_week'] = df.index.dayofweek  # 0=Monday
df['month'] = df.index.month

# Time arithmetic
df['time_since_start'] = df.index - df.index[0]
df['seconds_elapsed'] = df['time_since_start'].dt.total_seconds()

# Create datetime from frame numbers
fps = 30  # frames per second
start_time = pd.Timestamp('2024-01-15 10:00:00')
df['time'] = start_time + pd.to_timedelta(
    df['frame'] / fps, unit='s')
```

## Time-aware data structures

**DatetimeIndex benefits:**

- Slice with string dates: `df['2024-01']`
- Automatic date arithmetic
- Time-aware plotting (proper x-axis labels)
- Resampling and frequency conversion
- Easy filtering by time ranges

**Common datetime attributes:**

- `.year`, `.month`, `.day`
- `.hour`, `.minute`, `.second`
- `.dayofweek` (0=Monday, 6=Sunday)
- `.dayofyear` (1-366)
- `.quarter` (1-4)

**Time deltas:**

- `pd.Timedelta('1 day')`
- `pd.Timedelta(hours=2, minutes=30)`
- `pd.to_timedelta(df['seconds'], unit='s')`

**Why datetime indices?** Time-aware slicing, automatic alignment, resampling, date arithmetic, proper plotting

# Signal Processing

```python
import pandas as pd

# Resampling (requires DatetimeIndex)
# Downsample: high → low resolution
df_hourly = df.resample('1H').mean()      # Hourly avg
df_daily = df.resample('1D').sum()        # Daily sum
df_30s = df.resample('30S').agg({         # Custom agg
    'value': 'mean',
    'event': 'sum'
})

# Upsample: low → high resolution
df_upsampled = df.resample('100ms').ffill()  # Forward fill
df_interpolated = df.resample('100ms').interpolate()

# Common frequency strings
# 'S' = seconds, 'T'/'min' = minutes, 'H' = hours
# 'D' = days, 'W' = weeks, 'M' = months
# Can add numbers: '5T' = 5 minutes, '30S' = 30 seconds

# Rolling windows (moving statistics)
# Window specified in samples OR time
df['rolling_mean'] = df['signal'].rolling(
    window=100).mean()

# Time-based windows (requires DatetimeIndex)
df['rolling_mean_1s'] = df['signal'].rolling(
    window='1S').mean()

# Rolling with custom functions
df['rolling_std'] = df['signal'].rolling(50).std()
df['rolling_max'] = df['signal'].rolling(50).max()
df['rolling_sum'] = df['signal'].rolling(50).sum()
```

## Time-aware data structures

**Resampling aggregations:**

- `.mean()`, `.median()`, `.std()`
- `.sum()`, `.count()`
- `.min()`, `.max()`, `.first()`, `.last()`
- `.agg()` for custom/multiple aggregations

**Rolling window options:**

- `window`: Size (samples or time string)
- `center=True`: Center window on current point
- `min_periods`: Minimum observations required
- `win_type`: Window function (e.g., 'gaussian')

**Common patterns:**

- **Smoothing**: `rolling(50).mean()`
- **Volatility**: `rolling(100).std()`
- **Running total**: `rolling(20).sum()`
- **Z-score**: `(x - rolling_mean) / rolling_std`

**Resampling:** Change temporal resolution (downsample: reduce data points, upsample: increase data points)
**Rolling windows:** Compute statistics over moving time windows

# Signal Processing

```python
# Shift: move data forward/backward in time
df['previous_value'] = df['signal'].shift(1)   # Lag
df['next_value'] = df['signal'].shift(-1)      # Lead

# Difference: current - previous
df['change'] = df['signal'].diff()  # First difference
df['change_2'] = df['signal'].diff(2)  # 2-step difference

# Percentage change
df['pct_change'] = df['signal'].pct_change()
# Equivalent to: (current - previous) / previous

# Cumulative sum
df['cumulative'] = df['change'].cumsum()

# Velocity from position
df['velocity'] = df['position'].diff() / dt
# dt = time step (e.g., 1/fps for video)

# Acceleration from velocity
df['acceleration'] = df['velocity'].diff() / dt

# Smooth derivative (Savitzky-Golay)
from scipy import signal
velocity_smooth = signal.savgol_filter(
    df['position'], window_length=11, polyorder=2,
    deriv=1, delta=dt  # First derivative
)

# Example: Animal tracking
fps = 30
dt = 1 / fps
df['x_vel'] = df['x'].diff() / dt
df['y_vel'] = df['y'].diff() / dt
df['speed'] = np.sqrt(df['x_vel']**2 + df['y_vel']**2)
```

## Time-aware data structures: *change*

**Key methods:**

- `.shift(n)`: Move data by n positions (positive = lag, negative = lead)
- `.diff(n)`: Current value - value n positions ago
- `.pct_change()`: Percentage change from previous
- `.cumsum()`: Running cumulative sum

⚠️ **Important notes:**

- `.diff()` introduces NaN at start (no previous value)
- `.shift()` introduces NaN at edges
- Derivatives amplify noise - smooth first!
- Remember to divide by time step for proper units

**Computing derivatives properly:**

- **Simple**: `.diff() / dt` (noisy)
- **Better**: Smooth first, then differentiate
- **Best**: Savitzky-Golay with `deriv=1` (smooths and differentiates simultaneously)

**Essential operations for analyzing change over time:** Compare to previous/next values, calculate differences, compute velocities and accelerations

# Data Analysis Best Practices

Workflows for reliable, reproducible analysis

### 1. Start with exploration

- `df.head()`, `.info()`, `.describe()`
- Check data types and missing values
- Quick plots to understand distributions
- Identify outliers and anomalies early

### 2. Keep data tidy (long format)

- One observation per row
- One variable per column
- Easier for groupby, plotting, statistics
- Use `melt()` to convert wide → long

### 3. Document your cleaning steps

- Comment why you remove/impute data
- Track filtering decisions
- Record parameter choices (thresholds, etc.)
- Makes analysis reproducible and auditable

### 4. Chain operations with method chaining

- More readable:
  `df.filter().groupby().mean()`
- Avoids intermediate variables
- Use `\` for line breaks in long chains

# Data Analysis Best Practices

Workflows for reliable, reproducible analysis

### 5. Verify results at each step

- Check `df.shape` before/after operations
- Sanity check aggregations with plots
- Compare subset totals to originals
- Use `assert` statements for critical checks

### 6. Avoid loops when possible

- Vectorized operations are 10-100× faster
- Use `.apply()` only when necessary
- Prefer pandas built-in methods
- GroupBy handles most "per-group" operations

### 7. Save intermediate results

- Save cleaned data: `df.to_csv('cleaned.csv')`
- Avoid re-running expensive operations
- Parquet format for large datasets (faster, smaller)
- Version your data files

### 8. Use meaningful variable names

- `reaction_times` not `df2`
- `control_group` not `g1`
- Future you will thank present you

# Data Analysis Best Practices

⚠️ Don't!

## ❌ 1. Don't live in notebooks

- Notebooks encourage messy, non-linear code
- Hard to version control, test, and reuse
- Cell execution order creates hidden dependencies
- Use notebooks for exploration, scripts for production

## ❌ 2. Don't write monolithic scripts

- Break code into functions with clear purposes
- Group related functions into modules
- Max ~50 lines per function, ~300 per file
- Makes debugging and testing much easier

## ❌ 3. Don't copy-paste code

- Repeating code = repeating bugs
- Write a function if you do it twice
- DRY: Don't Repeat Yourself
- Changes need to happen in one place only

## ❌ 4. Don't code blindly

- If output looks weird, **stop** and investigate
- Use `print()`, plots, debugger liberally
- Check intermediate results, not just final output
- Silent failures create garbage results

## ❌ 5. Don't cargo-cult code

- Don't run code you don't understand
- Read documentation, not just Stack Overflow
- Ask colleagues to explain unclear methods
- Understanding > speed

## ❌ 6. Don't ignore warnings

- Warnings indicate potential problems
- `FutureWarning` = your code will break soon
- `SettingWithCopyWarning` = you're doing it wrong
- Fix the root cause, don't suppress

# Data Analysis Best Practices

⚠️ Don't!

### ❌ 4. Don't code blindly

### ❌ 2. Don't write monolithic scripts

### ❌ 5. Don't cargo-cult code

- Don't run code you don't understand

### ❌ 7. Don't modify original data

- Always work on copies: `df_clean = df.copy()`
- Keep raw data immutable
- You'll need to revisit cleaning decisions
- Inplace operations are rarely worth it

### ❌ 8. Don't trust your memory

- Write down what each column means
- Document units, ranges, encoding schemes
- Comment non-obvious filtering decisions
- You'll forget in 2 weeks, guaranteed

# Typical Analysis Workflow

```
# 1. LOAD DATA
df = pd.read_csv('experiment_data.csv')

# 2. INITIAL EXPLORATION
print(df.head())
print(df.info())
print(df.describe())
print(f"Missing values:\n{df.isnull().sum()}")

# 3. DATA CLEANING
# Handle missing values
df = df.dropna(subset=['subject_id', 'condition'])
df['reaction_time'].fillna(df['reaction_time'].median(), inplace=True)

# Remove outliers (RT < 100ms or > 2000ms likely errors)
df = df[(df['reaction_time'] > 100) & (df['reaction_time'] < 2000)]

# Convert types
df['condition'] = df['condition'].astype('category')
df['trial_date'] = pd.to_datetime(df['trial_date'])

# 4. CREATE DERIVED VARIABLES
df['log_rt'] = np.log(df['reaction_time'])
df['accuracy'] = (df['response'] == df['correct_answer']).astype(int)

# 5. AGGREGATE AND SUMMARIZE
summary = df.groupby(['subject', 'condition']).agg(
    mean_rt=('reaction_time', 'mean'),
    mean_accuracy=('accuracy', 'mean'),
    n_trials=('trial_id', 'count')
).reset_index()

# 6. STATISTICAL ANALYSIS
control = summary[summary['condition'] == 'control']['mean_rt']
treatment = summary[summary['condition'] == 'treatment']['mean_rt']
t_stat, p_value = stats.ttest_ind(control, treatment)
print(f"Treatment effect: t={t_stat:.2f}, p={p_value:.4f}")

# 7. VISUALIZATION
df.boxplot(column='reaction_time', by='condition')
plt.title('Reaction Time by Condition')
plt.suptitle('')  # Remove default title
plt.ylabel('Reaction Time (ms)')
plt.show()

# 8. SAVE RESULTS
summary.to_csv('analysis_results.csv', index=False)
print("Analysis complete!")
```

Putting it all together

**The 8-step workflow:**

1. **Load** - Read data from files
2. **Explore** - Understand structure, check for issues
3. **Clean** - Handle missing values, outliers, types
4. **Transform** - Create derived variables
5. **Aggregate** - Summarize by groups
6. **Analyze** - Statistical tests, correlations
7. **Visualize** - Plots for interpretation
8. **Save** - Export results for reporting

# Summary and Next Steps

What we covered today

- **Pandas fundamentals**
  - DataFrames and Series
  - Reading data (CSV, Excel)
  - Data selection and indexing
- **Data cleaning**
  - Missing data handling
  - Outlier detection
  - Type conversions
- **Data transformation**
  - Creating new columns
  - String and categorical operations
- **Grouping and aggregation**
  - GroupBy operations
  - Pivot tables
- **Reshaping and combining**
  - Long/wide format conversion
  - Concatenating and merging
- **SciPy for statistics**
  - Statistical tests (t-test, ANOVA, etc.)
  - Correlation analysis
  - Distribution fitting
- **Signal processing**
  - Filtering and smoothing
  - Peak detection
- **Timeseries operations**
  - DateTime handling
  - Resampling and rolling windows
  - Shift, diff, derivatives
- **Visualization basics**
  - Pandas built-in plotting

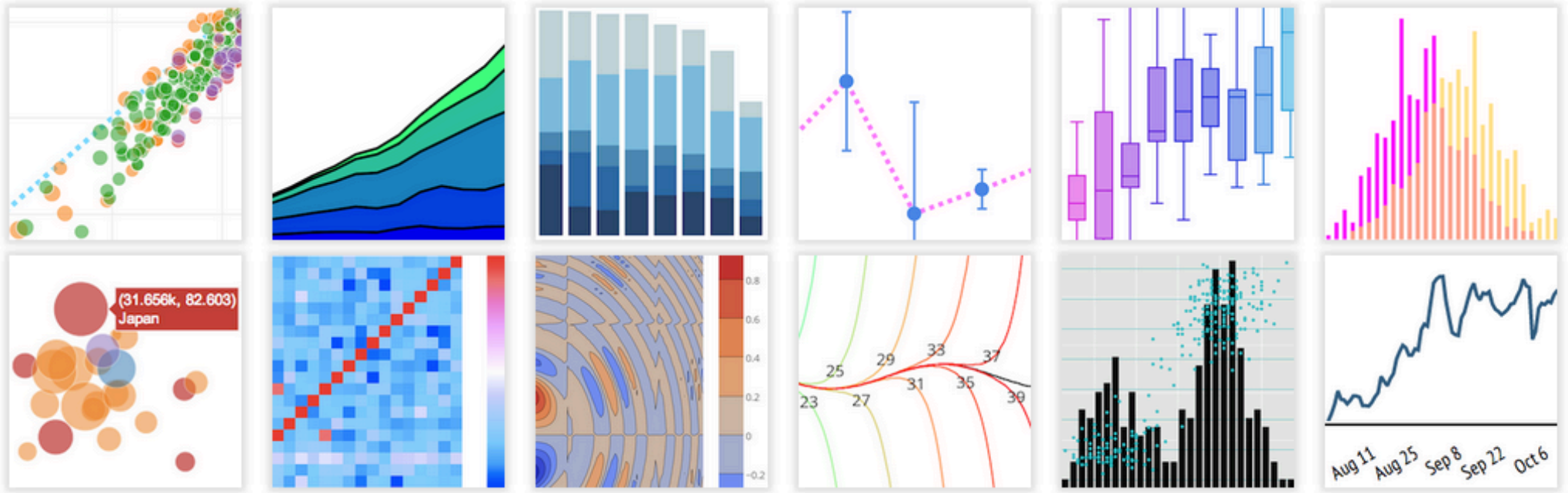
# Summary and Next Steps

- Advanced visualization with **Seaborn**
- Interactive plots with **Plotly**
- Creating publication-quality figures

seaborn

plotly

Next Week



# Summary and Next Steps

```python
import pandas as pd
import matplotlib.pyplot as plt

# Line plot (default)
df['signal'].plot()
df.plot(x='time', y='value')

# Multiple columns
df[['gene_A', 'gene_B', 'gene_C']].plot()

# Scatter plot
df.plot.scatter(x='dose', y='response')

# Histogram
df['reaction_time'].plot.hist(bins=30)
df.hist(figsize=(12, 8))  # All numeric columns

# Box plot
df.boxplot(column='expression', by='condition')
df[['control', 'treated']].plot.box()

# Bar plot
df.groupby('condition')['accuracy'].mean().plot.bar()

# Area plot (stacked)
df.plot.area()

# Kernel Density Estimate
df['values'].plot.kde()

# Customization
df['signal'].plot(
    figsize=(10, 4),
    title='Neural Signal Over Time',
    xlabel='Time (s)',
    ylabel='Amplitude (mV)',
    color='navy',
    linewidth=2,
    grid=True
)
```

**Common plot types:**

- `.plot()` or `.plot.line()` - Timeseries, trends
- `.plot.scatter()` - Relationships between variables
- `.plot.hist()` - Distributions
- `.plot.box()` - Compare distributions, outliers
- `.plot.bar()` - Categorical comparisons
- `.plot.barh()` - Horizontal bar chart
- `.plot.kde()` - Smooth density estimate
- `.plot.area()` - Stacked area chart

**Key parameters:**

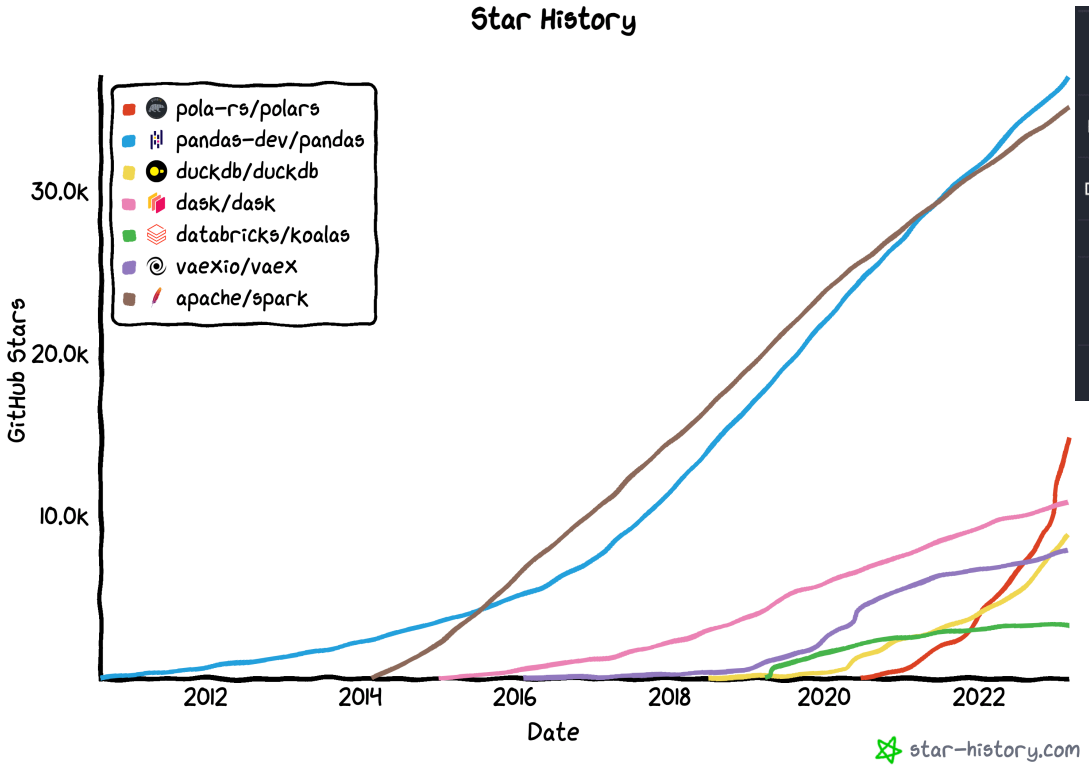
- `figsize=(width, height)` - Figure size in inches
- `title, xlabel, ylabel` - Labels
- `color, linewidth, alpha` - Styling
- `grid=True` - Add grid lines
- `legend=True/False` - Show/hide legend
- `subplots=True` - Separate subplot per column

**When to use pandas plotting:**

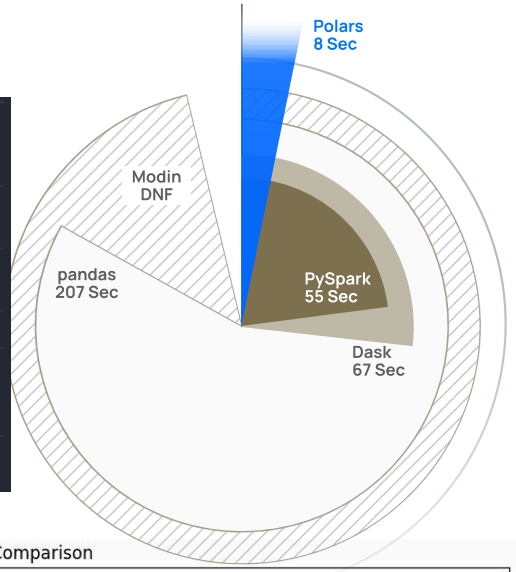
- ✅ Quick data exploration in Jupyter
- ✅ Rapid prototyping and sanity checks
- ✅ Simple plots for yourself
- ❌ Publication-quality figures (use matplotlib/seaborn)
- ❌ Complex multi-panel layouts
- ❌ Interactive plots (use plotly - next week!)

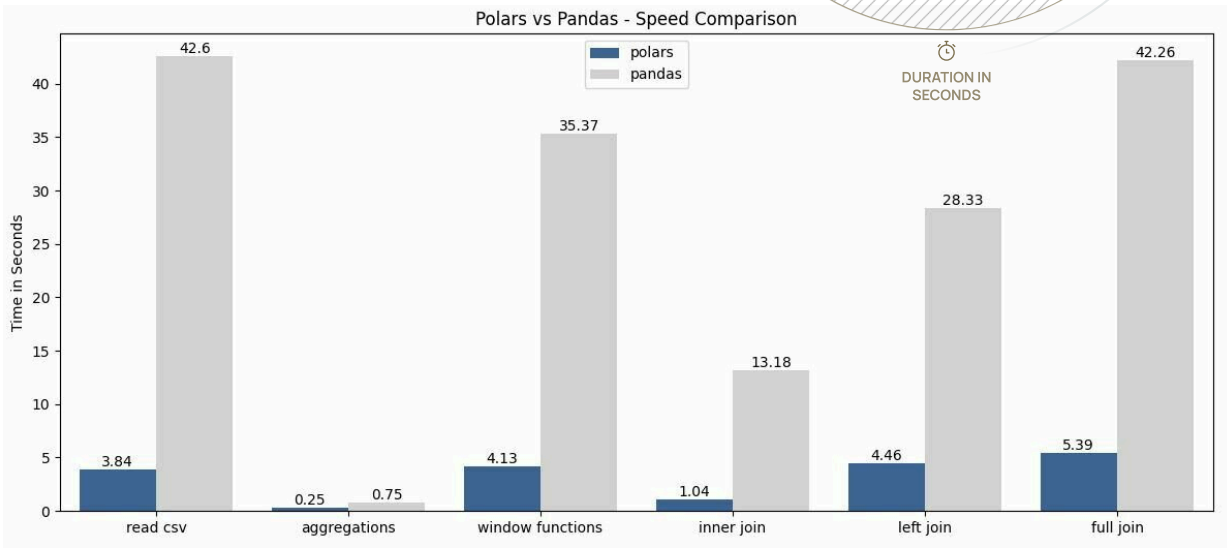
# Summary and Next Steps

## Side note: *pandas alternatives*




| Tool Name | Key Feature | Filter | Group By | Join | Window Func |
|---|---|---|---|---|---|
| Pandas | widely adopted data analysis and manipulation tool | 0.416s | 0.600s | 4.159s | 17.465s |
| DuckDB | OLAP in memory Database, SQL friendly | 0.410s | 0.122s | 3.364s | 6.466s |
| Polars | Rust, Arrow, multi cores | 0.0523s | 0.0808s | 1.343s | 2.705s |
| Vaex | Out-of-Core, Data Visualization, optimization on group by and joins | 0.006s | 2.987s | 4.224s | N/A |
| Modin | Scale with one line of changes | 0.828s | 1.211s | 1.389s | 15.635s |




Polars

# Thank you!





Course materials adapted from:

- Moritz Helias
- Abigail Morrison
- Yury Zaitsev
- Renato Duarte





With some help from Claude Sonnet 4.5

Claude